

Optimización de Rutas Urbanas mediante Algoritmo Genético híbrido con Colonia de Hormigas

Grupo: Extensiones de Machine Learning – UPCT

Miembros Alfonso Guzmán Ortiz, Mauro Martínez Cazaux

1. Introducción y Objetivos

1.1 Contexto del problema

La optimización de rutas urbanas es un problema clásico en ámbitos logísticos, diseño de rutas de servicio o transporte de mercancías. Estos escenarios se pueden modelar como variantes del Problema del Viajante (TSP), donde se busca encontrar la ruta de coste mínimo que visita un conjunto de ubicaciones exactamente una vez y vuelve (o no) al origen.

En entornos reales, el coste de desplazamiento entre dos puntos no viene dado por la distancia Euclídea, sino por la distancia real por carretera en una red viaria. Esto hace que el problema sea más complejo y, en general, no simétrico.

Este proyecto aborda un TSP urbano realista y dinámico sobre una selección de ubicaciones distribuidas por España, utilizando distancias reales por carretera obtenidas mediante APIs de cartografía digital.

1.2 Motivación y objetivos del proyecto

La motivación principal es diseñar y evaluar un sistema de optimización de rutas urbanas que combine datos geoespaciales reales con técnicas metaheurísticas avanzadas. Se persigue:

1. Integrar datos de OpenStreetMap y OpenRouteService para trabajar con una matriz de distancias realista.
2. Implementar y evaluar por separado un Algoritmo Genético (GA) y un Algoritmo de Colonia de Hormigas (ACO)
3. Proponer e implementar un enfoque híbrido GA—ACO que aproveche las fortalezas de ambos métodos (exploración global y explotación local)
4. Comparar los algoritmos en términos de distancia total recorrida (km), tiempo de cómputo (s), estabilidad de las soluciones ante múltiples ejecuciones.
5. Visualizar las rutas obtenidas sobre un mapa interactivo para facilitar la interpretación de resultados

1.3 Tecnologías utilizadas

Para desarrollar el proyecto se han empleado las siguientes tecnologías y librerías:

1. Jupyter Notebook para combinar código, resultados y visualización
2. Un entorno virtual para aislar dependencias y evitar incompatibilidades entre librerías
3. *OpenStreetMap* como fuente de datos cartográficos open-source

4. *Nominatim* (geocodificación), mediante la librería *geopy*, para convertir direcciones en coordenadas
5. *OpenRouteService* para el cálculo de distancias por carretera y obtener la matriz de distancias D
6. Librerías *numpy* para el manejo eficiente de matrices, *time* para medir tiempos de ejecución, *folium* para generar los mapas de visualización

2. Preparación de datos – Obtención de coordenadas geográficas

1.1 Obtención de la API de OpenStreetMap

Para resolver el problema de optimización de rutas urbanas, es necesario disponer de coordenadas geográficas precisas de las ubicaciones a visitar. En este proyecto, se optó por utilizar la API de geocodificación de OpenStreetMap (Nominatim), una herramienta de código abierto que permite convertir direcciones físicas en coordenadas geográficas (latitud y longitud).

Esta elección se basa en que OpenStreetMap es una plataforma colaborativa y gratuita que provee datos geoespaciales de calidad sin las restricciones y costes asociados a servicios comerciales como Google Maps.

La API de Nominatim es un servicio de geocodificación que permite obtener coordenadas a partir de direcciones y viceversa. A diferencia de otros servicios, Nominatim es completamente open-source, lo que permite su uso gratuito y sin limitaciones de acceso para proyectos de pequeña escala.

1.2 Obtención de Coordenadas de Ubicaciones

Una vez seleccionamos las ubicaciones de interés utilizamos la librería de *geopy* para interactuar con la API de *Nominatim* a través de un script que recibe una lista de direcciones textuales y retorna una lista de coordenadas (latitud, longitud) de cada ubicación.

1.3 Uso de Coordenadas en el Proyecto

Utilizamos las coordenadas obtenidas mediante *Nominatim* para construir la lista de puntos que se proyectarán sobre la red viaria, para generar la matriz de distancias D entre todos los pares de ubicaciones y visualizar posteriormente las rutas.

En el núcleo de los algoritmos GA y ACO no se trabaja directamente con coordenadas, sino con los índices de nodos. La relación entre índices y coordenadas permite reconstruir la ruta y proyectarla sobre la red viaria.

1.4 Conclusión Fase 1

En esta fase, se ha conseguido establecer la infraestructura necesaria para obtener coordenadas geográficas reales utilizando la API Nominatim de OpenStreetMap. Estos datos servirán como la base sobre la que se construirán las rutas a optimizar. La elección de una solución open-source como OSM también facilita el acceso

gratuito y sostenible a los datos geoespaciales, lo que permite llevar a cabo el proyecto sin restricciones externas.

3. Construcción de la matriz de distancias por carretera (OpenRouteService)

1.1 Objetivo

En lugar de generar una población inicial explícita, optamos por partir de una matriz de distancias $n \times n$, donde cada entrada $D[i, j]$ es la distancia real por carretera entre los puntos i y j calculada con ORS. Esta matriz es la base para ACO y GA

1.2 Procedimiento

Geocodificamos 20 ubicaciones distribuidas por toda España.

Calculamos D mediante la API `distance_matrix/directions` de ORS incorporando:

1. *Snapping* a la red viaria y radios crecientes para puntos difíciles
2. Asimetría natural ($D[i, j] \neq D[j, i]$) para sentidos únicos y giros
3. Evitar límite impuesto por API de 6000 km construyendo tramo a tramo cuando es necesario

Se almacena la matriz D para evitar recalcular distancias en cada ejecución del notebook.

1.3 Observaciones

El problema resultante es un TSP dirigido definido sobre la matriz D . Para facilitar la comparación entre algoritmos, se fija un nodo de inicio común ($START_IDX$). De este modo todas las rutas comienzan en el mismo punto y las comparaciones entre algoritmos son coherentes.

1.4 Conclusión Fase 3

Se dispone de una matriz D robusta y realista que captura la topología de la red viaria española. A partir de esta matriz, ACO y GA operan sobre índices evaluando el coste de una ruta como la suma de aristas en D .

4. Optimización de Rutas mediante el Algoritmo Genético

1.1 Introducción al Algoritmo Genético

El Algoritmo Genético (GA) es una metaheurística inspirada en los principios de la evolución natural: *selección*, *cruce* y *mutación*. Trabaja con una población de soluciones candidatas que evolucionan generación tras generación, con el objetivo de mejorar su aptitud.

En este proyecto, cada individuo representa una ruta candidata para el TSP definido por la matriz D . El objetivo del GA es encontrar la permutación de nodos que minimice la distancia total recorrida.

1.2 Representación y evaluación

Un individuo es una permutación de índices de nodos en la que el nodo inicial se mantiene fijo y el resto de posiciones se permutan libremente.

La población inicial se genera a partir de permutaciones aleatorias de los índices, de manera que se cubra de forma amplia el espacio de búsqueda desde el comienzo.

1.3 Operadores del Algoritmo Genético

Estos son los operadores que utilizará el algoritmo para evolucionar la población

- **Selección:** Se aplica selección por torneo de tamaño k . Se eligen al azar k individuos de la población y se selecciona el de mejor coste como progenitor.
- **Cruce:** Usamos un cruce de tipo *Order Crossover (OX)*. Se selecciona un segmento de la ruta como progenitor, se preserva dicho segmento y se completa la ruta con el orden relativo del otro progenitor.
- **Mutación:** Se aplica una mutación por intercambio de dos posiciones de la ruta (exceptuando el nodo inicial). Esta mutación ayuda a explorar nuevas combinaciones de nodos y evitar estancarse en óptimos locales
- **Elitismo:** Un número fijo de los mejores individuos de cada generación se copia directamente a la siguiente sin alteración, garantizando que las mejores soluciones encontradas no se pierdan por efecto del cruce y mutación.

1.4 Implementación del Algoritmo Genético

El flujo del algoritmo es:

1. Inicialización de la población con permutaciones aleatorias
2. Evaluación del coste de cada individuo mediante la matriz D
3. Repetición durante un número fijo de generaciones
 - Selección de progenitores por torneo
 - Aplicación de cruce OX y mutación
 - Inserción de individuos *élite* en la nueva población
4. Mantenimiento de la mejor solución hallada a lo largo de todas las generaciones

Al finalizar, GA retorna la mejor ruta encontrada y su coste asociado.

1.5 Visualización de la Mejor Ruta

Una vez obtenido el mejor individuo, se traduce la permutación de índices a coordenadas geográficas, se reconstruye la ruta completa solicitando a ORS los tramos por carretera correspondientes. Generamos un mapa interactivo con *folium*, donde:

- Los puntos de interés se marcan con su orden de visita
- La ruta optimizada se representa como una polilínea sobre la red viaria
- El usuario puede hacer zoom y desplazar el mapa para analizar el trayecto

Este mapa se guarda en un archivo *.html* en el directorio de ejecución del programa.



1.6 Conclusión Fase 4

Se ha implementado y ejecutado un Algoritmo Genético capaz de optimizar rutas sobre la matriz de distancias D . A través de un proceso evolutivo guiado por la *selección*, *cruce*, *mutación* y *elitismo*, el GA es capaz de mejorar las soluciones iniciales aleatorias y proporcionar rutas de coste reducido. La visualización interactiva refuerza la interpretación de los resultados y su coherencia con el entorno urbano modelado.

5. Optimización de Rutas mediante Algoritmo de Colonia de Hormigas

1.1 Justificación del uso de ACO

El Algoritmo de Colonia de Hormigas (ACO) es una metaheurística de optimización inspirada en el comportamiento colectivo de las hormigas. En la naturaleza, las hormigas son capaces de encontrar caminos cortos entre el nido y una fuente de alimento depositando feromonas sobre las rutas que recorren. Las rutas más cortas acumulan más feromonas y se vuelven más atractivas, generando un mecanismo de optimización.

Este problema se ha trasladado al ámbito computacional para resolver problemas de optimización combinatoria, como el TSP. En el contexto de este proyecto, ACO resulta adecuado porque trabaja de forma natural con grafos de costes (matriz D) y combina exploración con explotación.

1.2 Implementación de ACO

El ACO se implementa en pasos secuenciales, emulando cómo las hormigas exploran y encuentran el mejor camino en un grafo de ubicaciones. A continuación, se describen las principales fases de la implementación:

1. Inicialización de las hormigas:

- Se define una matriz de feromonas τ inicialmente uniforme.
- Todas las hormigas comparten un nodo inicial fijo.

- Se calcula la matriz de visibilidad $1/D+e$

2. Construcción de rutas:

- Cada hormiga construye una ruta completa visitando todos los nodos una sola vez.
- La probabilidad de que una hormiga en el nodo i vaya al nodo j es proporcional a:

$$P(i \rightarrow j) \propto \tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$$

3. Evaluación de rutas

- La evaluación de cada ruta se realiza con:

$$coste(\pi) = \sum_k D_{\pi_k \pi_{k+1}}$$

4. Actualización de feromonas:

- Tras construir todas las rutas, se aplica evaporación global:

$$\tau \leftarrow (1 - p) \cdot \tau$$

Donde p es la tasa de evaporación

- Se realiza el depósito de feromonas por parte de las hormigas:

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{\text{hormigas que usan } (i,j)} \frac{Q}{coste(\pi)}$$

De modo que las rutas más cortas reciben más feromonas

5. Iteración y selección de la mejor ruta:

- El proceso se repite durante un número fijo de iteraciones
- Se mantiene y actualiza la mejor ruta global encontrada hasta el momento.

1.3 Parámetros clave de ACO:

- **Número de hormigas:** número de soluciones generadas por iteración.
- **α :** peso de las feromonas en la decisión de la siguiente arista.
- **β :** peso de visibilidad ($1 / \text{Distancia}$).
- **p :** tasa de evaporación de las feromonas.
- **Q :** Cantidad de feromonas depositadas por las hormigas.
- **Número de iteraciones:** Controla el tiempo de cómputo y el grado de refinamiento de las selecciones.

1.4 Visualización de las Rutas Generadas por ACO

Una vez que ACO ha convergido o alcanzado el número máximo de iteraciones, se visualiza la ruta sobre un mapa interactivo:

1. Se recupera la secuencia de coordenadas asociada a los índices de la ruta
2. Se solicita a ORS la geometría de la ruta por carretera usando el perfil *driving-car*
3. Se dibuja la polilínea en un mapa *folium*



1.5 Conclusión Fase ACO

En esta fase, hemos implementado el Algoritmo de Colonia de Hormigas (ACO) para optimizar las rutas entre las ubicaciones de interés. El algoritmo utiliza el comportamiento de las hormigas para explorar soluciones y mejorar las rutas a lo largo de varias iteraciones, guiado por el depósito de feromonas.

La mejor ruta generada por ACO fue visualizada sobre un mapa interactivo utilizando OpenRouteService para calcular las rutas por carretera y folium para la representación gráfica. Esto permitió comparar visualmente la ruta optimizada y entender cómo el algoritmo seleccionó el mejor camino entre las ubicaciones.

Con esta fase completada, el ACO ha proporcionado una solución óptima o casi óptima para el problema del Viajante de Comercio (TSP), que será comparada con los resultados obtenidos mediante el Algoritmo Genético (GA) en la siguiente fase.

6. Comparación de Resultados – GA vs ACO

1.1 Protocolo de comparación

Para comparar GA y ACO de forma justa se ha seguido el siguiente protocolo:

- **Matriz de costes común:** Ambos algoritmos utilizan la misma matriz D calculada con ORS
- **Mismo nodo de inicio:** Todas las rutas comienzan en el mismo nodo para asegurar comparabilidad directa
- **Métricas evaluadas:**
 - Métrica principal: Distancia total por carretera (en km), obtenida como la suma de los elementos de D a lo largo de la ruta
 - Métrica secundaria: Tiempo de cómputo (en s) por ejecución

- **Múltiples ejecuciones:** Se realizan varias ejecuciones independientes de cada algoritmo con las mismas configuraciones de parámetros para medir la estabilidad y detectar posibles estancamientos en óptimos locales

1.2 Resultados

En este apartado se recogen:

- Las distancias medias, mínimas y máximas obtenidas por GA y ACO
- Los tiempos de cómputo asociados a cada método
- Observaciones sobre:
 - La velocidad de convergencia
 - Sensibilidad a los parámetros
 - Robustez frente a múltiples soluciones

Algoritmo	Mejor ruta	Coste total (km)	Tiempo de ejecución (s)
GA	[0, 10, 19, 12, 7, 14, 13, 17, 16, 5, 15, 8, 3, 9, 6, 18, 11, 2, 4, 1]	4826.7	0.8438
ACO	[0, 12, 19, 9, 6, 18, 11, 7, 14, 13, 2, 17, 4, 1, 10, 8, 3, 15, 5, 16]	4371.23	2.2825

En la prueba realizada, ambos algoritmos fueron ejecutados sobre la misma matriz de distancias y con el mismo nodo inicial. Los resultados muestran diferencias claras tanto en calidad de solución como en tiempo de ejecución:

- **Calidad de solución:**
 - ACO obtuvo una ruta de 4371.23 km, mientras que GA alcanzó 4826.7 km.
 - Esto supone que ACO logró una ruta aproximadamente un 9.5% más corta respecto al resultado de GA
 - Es una diferencia coherente con la naturaleza de ACO, que tiende a explorar rutas prometedoras reforzando aristas con feromonas, lo que facilita converger hacia soluciones más refinadas
- **Tiempo de ejecución:**
 - GA completó su ejecución en 0.84 segundos mientras que ACO necesitó 2.28.
 - GA es más rápido porque genera un número fijo de individuos por generación y su coste computacional es lineal con respecto al tamaño de la solución
 - ACO es más costoso ya que cada hormiga construye una ruta completa por iteración, y además realiza evaporación y actualización de feromonas.

En conjunto, los resultados indican que GA es más eficiente computacionalmente (adecuado cuando se requieren soluciones rápidas) mientras que ACO ofrece soluciones de mayor calidad (útil cuando el tiempo de cómputo no es la limitación principal).

Estos resultados justifican la motivación del enfoque híbrido, que pretende combinar la rapidez y diversidad inicial del GA y el refinamiento profundo del ACO en su fase de explotación.

1.3 Visualización comparativa

Para apoyar la comparación cualitativa se realiza una visualización dual:



Con esta visualización conjunta observamos donde divergen las rutas propuestas por cada algoritmo, que algoritmo tiende a explotar determinados caminos viarios y las diferencias en el orden de visita de ciertas ubicaciones.

7. Propuesta e implementación del algoritmo híbrido GA—ACO

1.1 Introducción

Tras analizar por separado las prestaciones de GA y ACO, se propone un enfoque híbrido GA—ACO que busque combinar la capacidad exploratoria del Algoritmo Genético y la capacidad explotadora del Algoritmo de Colonia de Hormigas.

La idea es utilizar el GA como una fase inicial de exploración amplia del espacio de soluciones y aprovechar esa información para guiar el proceso de ACO mediante una matriz de feromonas inicial informada.

1.2 Fundamentación teórica

El GA genera una gran diversidad de soluciones gracias a sus operadores de cruce y mutación. Esto permite explorar muchas configuraciones de rutas y descubrir regiones prometedoras del espacio de búsqueda.

ACO, por su parte, concentra su esfuerzo en refinar rutas que han demostrado ser buenas, reforzándolas mediante el depósito de feromonas y reduciendo progresivamente la exploración de opciones peores.

La combinación de ambos en un esquema secuencial puede ofrecer una exploración global eficaz y un refinamiento local eficiente en torno a las mejores soluciones iniciales.

1.3 Implementación propuesta

En la implementación se han reutilizado exactamente las mismas funciones básicas empleadas en los algoritmos independientes.

Este enfoque se estructura en dos fases principales:

1. Fase inicial – GA exploratorio

Se ejecuta el GA con un número moderado de generaciones y se lanza varias veces de forma independiente, obteniendo así un conjunto de rutas de buena calidad

2. Fase de refinamiento – ACO guiado por feromonas iniciales

Se construye una matriz de feromonas iniciales (τ^0) a partir del *elite set*:

- Las aristas que aparecen con mayor frecuencia en las mejores rutas reciben mayor cantidad de feromonas iniciales
- De este modo, la información aprendida por el GA se “inyecta” en el ACO

Se ejecuta el ACO de forma estándar, pero partiendo de la matriz τ^0 en lugar de una matriz uniforme:

- Las hormigas tienen una mayor probabilidad de explorar rutas afines a las sugeridas por el GA.
- La evaporación y el depósito de nuevas feromonas permiten seguir adaptando la búsqueda y escapar de posibles sesgos

En términos de código, el híbrido se implementa como una orquestación secuencial:

1. Ejecutar el GA varias veces y construir el *elite set*
2. Traducir el *elite set* a una matriz de feromonas inicial
3. Llamar al ACO reutilizando la misma función que en la versión independiente, pero pasando la matriz τ^0 como *tau_init*

1.4 Ventajas esperadas

El enfoque híbrido presenta varias ventajas potenciales frente a las ejecuciones independientes de GA o ACO:

- **Mayor estabilidad de resultados:** al combinar información de múltiples ejecuciones de GA con el refinamiento de ACO, se espera reducir la variabilidad entre ejecuciones
- **Menor número de iteraciones totales** para alcanzar soluciones de calidad, ya que ACO parte de una buena base proporcionada por el GA.

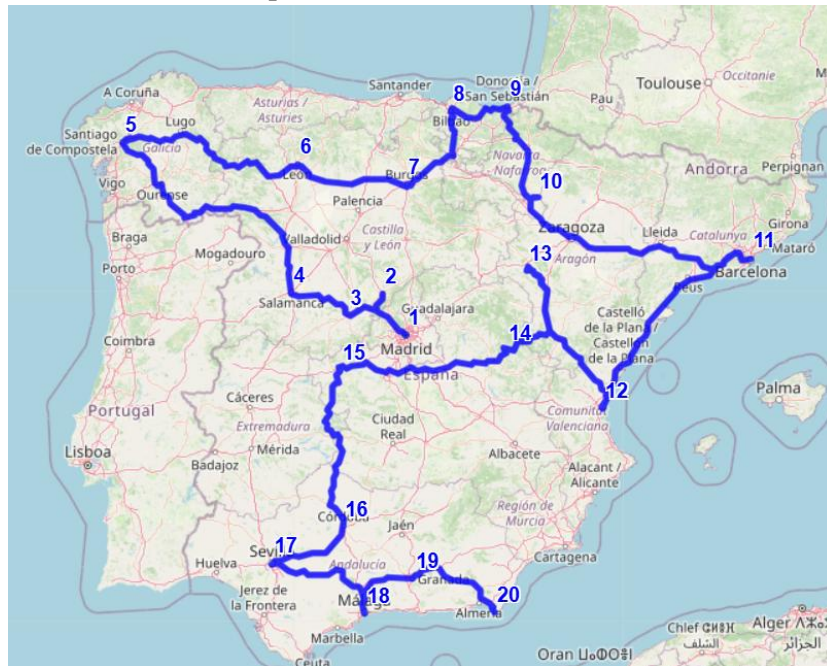
- **Mejor equilibrio entre exploración y explotación:**
 - GA aporta diversidad y evita depender únicamente del refuerzo de rutas locales
 - ACO se encarga de consolidar y mejorar las rutas más prometedoras
- **Escalabilidad:** El esquema GA—ACO puede extenderse a problemas con más ubicaciones u otras restricciones sin modificar el núcleo de los algoritmos, únicamente ajustando parámetros.

1.5 Limitaciones

Este enfoque presenta algunas limitaciones:

- **Mayor complejidad de implementación y ajuste:** Es necesario configurar y calibrar correctamente dos algoritmos en cascada, lo que incrementa el número de parámetros a sintonizar.
- **Coste computacional adicional:** La fase exploratoria de GA añade un sobre coste de cómputo antes de la fase ACO. Aunque esto se puede compensar reduciendo el número de iteraciones de ACO, sigue existiendo un equilibrio que debe ajustarse según el problema.
- **Dependencia del *elite set*:** Si el *elite set* obtenido por GA es de baja calidad o poco diverso, la matriz de feromonas inicial puede sesgar en exceso la búsqueda de ACO hacia regiones subóptimas.
- **Sensibilidad a la escala del problema:** Para instancias mucho más grandes, puede ser necesario rediseñar parcialmente la estrategia de hibridación

1.6 Visualización del mapa



1.7 Comparativa entre modelos independientes e hibridación

Además de evaluar GA y ACO por separado, se analizó el rendimiento del algoritmo híbrido GA–ACO, en el cual el GA actúa como fase exploratoria y el ACO refina las rutas partiendo de una matriz de feromonas inicial informada por el conjunto élite.

Algoritmo	Mejor ruta	Coste total	Tiempo ejecución
GA	[0, 10, 19, 12, 7, 14, 13, 17, 16, 5, 15, 8, 3, 9, 6, 18, 11, 2, 4, 1]	4826.70	0.8438
ACO	[0, 12, 19, 9, 6, 18, 11, 7, 14, 13, 2, 17, 4, 1, 10, 8, 3, 15, 5, 16]	4371.23	2.2825
Híbrido GA—ACO	[0, 12, 19, 10, 8, 3, 15, 5, 16, 4, 1, 2, 13, 14, 7, 11, 18, 6, 9, 17]	4304.30	3.76

1. Calidad de solución

El híbrido obtuvo un coste de 4304.29 km, lo que supone una mejora del 10.8 % respecto a GA y del 1.53 % respecto a ACO

Este resultado es especialmente relevante porque el híbrido supera tanto a GA como a ACO, cumpliendo plenamente la motivación teórica: partir de una exploración amplia del GA y refinarla con el ACO permite alcanzar una calidad de solución superior a cualquiera de los métodos por separado.

2. Tiempo de ejecución

El híbrido tardó 3.76 segundos frente a 0.84 de GA y 2.28 de ACO.

Este coste es esperable ya que ejecuta dos procesos consecutivos.

En términos prácticos, si el criterio prioritario es obtener la mejor solución posible, el híbrido es la opción recomendada.

3. Comportamiento global del híbrido

El nuevo resultado confirma varias ventajas clave del enfoque GA–ACO:

- **Convergencia más rápida:** ACO, al partir de una matriz de feromonas informada por el GA, tiende a reforzar de inmediato aristas prometedoras, evitando fases iniciales de exploración “a ciegas”.
- **Menor variabilidad:** El híbrido reduce el riesgo de que GA o ACO, en ejecuciones individuales, caigan en regiones subóptimas del espacio de búsqueda.
- **Sinergia real:**
 - GA aporta diversidad y exploración global.
 - ACO aporta refinamiento local y consolidación de rutas buenas.

- La combinación obtiene una solución que ninguno de los dos algoritmos logra por separado en esta instancia concreta.

1.8 Conclusión

El enfoque híbrido GA–ACO demuestra ser la estrategia más eficaz entre las evaluadas:

- Supera ampliamente a GA en calidad de solución.
- Supera a ACO puro en esta ejecución, con una mejora del 1.5 %.
- Mantiene un tiempo de ejecución moderado y asumible para entornos en los que la calidad es más importante que unos segundos extra de cálculo.

En conjunto, el híbrido logra la mejor solución del estudio, validando la hipótesis de que la exploración genérica del GA combinada con el refinamiento del ACO permite obtener resultados superiores y más estables.

8. Conclusiones

1.1 Conclusiones específicas del proyecto

A lo largo del proyecto se ha desarrollado un sistema completo para la optimización de rutas urbanas que integra datos geoespaciales reales obtenidos mediante OpenStreetMap y OpenRouteService, una modelización del problema TSP dirigido sobre una matriz de distancias realista y tres enfoques de optimización (ACO, GA e híbrido)

Los resultados muestran que tanto GA como ACO son capaces de encontrar rutas de calidad sobre la matriz D , mejorando ampliamente las soluciones aleatorias, la visualización interactiva sobre mapa facilita la interpretación de las diferencias entre rutas y el análisis cualitativo de las soluciones y que el enfoque híbrido permite incorporar información global aprendida por el GA en el proceso de refinamiento local de ACO, ofreciendo un marco flexible que puede adaptarse a otras instancias del problema.

1.2 Trabajo futuro

Como posibles extensiones del proyecto se plantean:

- Incrementar el número de ubicaciones y estudiar el comportamiento de los algoritmos en problemas de mayor escala como por ejemplo rutas entre países.
- Introducir restricciones adicionales como ventanas temporales, capacidad de vehículos, prioridades de entrega o tiempo máximo de ruta.
- Integrar información dinámica como el tráfico en tiempo real siempre que las APIs utilizadas lo permitan.